

Exercise 1.1.1

Consider a floating-point set $\mathbb{F}$ defined by (1.1.1) and (1.1.2) with $d = 4$.

(a) How many elements of $\mathbb{F}$ are there in the real interval $[1/2, 4]$, including the endpoints?

With $d = 4$, we have $z \in [0: 2^d - 1 = 15]$ and thus $f \in 2^{-d}z = \{z/16 \mid \forall z\}$.

With $e = -1$, we have $(1 + f)2^e = \{16/32, 17/32, \dots 31/32\}$, 16 elements in $[1/2, 1)$.

With $e = 0$, we have $(1 + f)2^e = \{16/16, 17/16, \dots 31/16\}$, 16 elements in $[1, 2)$.

With $e = 1$, we have $(1 + f)2^e = \{16/8, 17/8, \dots 31/8\}$, 16 elements in $[2, 4)$.

Add $4 = (1 + 0/16)2^2$ to get 49 elements total in the real interval $[1/2, 4]$.

(b) What is the element of $\mathbb{F}$ closest to the real number $1/10$? (Hint: Find the interval $[2^n, 2^{n+1})$ that contains $1/10$, then enumerate all the candidates in $\mathbb{F}$.)

$e = -4$ generates FPs on $1/10 \in [1/16, 1/8)$.

With $e = -4$, we have $(1 + f)2^e = 16/256, 17/256, \dots, 31/256$, of which $26/256 = (1 + 10/16)2^{-4}$ is closest.

(c) What is the smallest positive integer not in $\mathbb{F}$? (Hint: For what value of the exponent does the spacing between floating-point numbers become larger than 1?)

$e = 4$ generates $\{16, 17, \dots 31\}$, every integer on $[16, 32)$.

$(1 + 0/16)2^5 = 32$, and $(1 + 1/16)2^5 = 34$, skipping 33. 33 is the smallest positive integer not in $\mathbb{F}$.

Exercise 1.1.2

Prove that (1.1.5) is equivalent to (1.1.6). This means showing first that (1.1.5) implies (1.1.6), and then separately that (1.1.6) implies (1.1.5).

(1.1.5) states $\frac{|\text{fl}(x) - x|}{|x|} \leq \frac{2^{n-d-1}}{2^n} = \frac{1}{2}\epsilon_M$, and (1.1.6) states $\text{fl}(x) = x(1 + \epsilon)$ for some $|\epsilon| \leq \frac{1}{2}\epsilon_M$.

(1.1.5) $\rightarrow$ (1.1.6):

$$\begin{aligned} \frac{|\text{fl}(x) - x|}{|x|} &= \left| \frac{\text{fl}(x) - x}{x} \right| \leq \frac{1}{2}\epsilon_M \\ \rightarrow -\frac{1}{2}\epsilon_M &\leq \frac{\text{fl}(x) - x}{x} \leq \frac{1}{2}\epsilon_M \end{aligned}$$

Now assume $x > 0$.

$$\begin{aligned} \rightarrow \text{fl}(x) - x &\leq x \frac{1}{2}\epsilon_M \\ \rightarrow \text{fl}(x) &\leq x(1 + \frac{1}{2}\epsilon_M) \end{aligned}$$

Now if we set $\text{fl}(x) = x(1 + \epsilon)$ where $\epsilon \leq \frac{1}{2}\epsilon_M$, we get exactly (1.1.6) for positive ϵ .

If we instead assume $x < 0$,

$$\begin{aligned} \rightarrow -x \frac{1}{2}\epsilon_M &\geq \text{fl}(x) - x \\ \rightarrow x(1 - \frac{1}{2}\epsilon_M) &\geq \text{fl}(x) \end{aligned}$$

Now if we set $\text{fl}(x) = x(1 + \epsilon)$ where $-\epsilon \leq \frac{1}{2}\epsilon_M$, we get exactly (1.1.6) for negative ϵ .

We can combine these into the given $|\epsilon| \leq \frac{1}{2}\epsilon_M$.

(1.1.6) $\rightarrow$ (1.1.5):

$$\begin{aligned} \text{fl}(x) &= x(1 + \epsilon), |\epsilon| \leq \frac{1}{2}\epsilon_M \\ \rightarrow -\frac{1}{2}\epsilon_M &\leq \epsilon \leq \frac{1}{2}\epsilon_M \end{aligned}$$

Now assume positive ϵ .

$$\rightarrow \text{fl}(x) \leq x(1 + \frac{1}{2}\epsilon_M)$$

We can walk back the algebraic manipulation shown in (1.1.5) $\rightarrow$ (1.1.6) to reach $\frac{|\text{fl}(x)-x|}{|x|} \leq \frac{1}{2}\epsilon_M$.

If we instead assume negative ϵ ,

$$\rightarrow x(1 - \frac{1}{2}\epsilon_M) \geq \text{fl}(x)$$

We can again walk back the algebraic manipulation to reach the same $\frac{|\text{fl}(x)-x|}{|x|} \leq \frac{1}{2}\epsilon_M$.

As (1.1.5) $\rightarrow$ (1.1.6) and (1.1.6) $\rightarrow$ (1.1.5), they are equivalent.

Exercise (1.2.1)

Use (1.2.6) to derive the relative condition numbers of the following functions appearing in Table 1.2.1.

(a) $f(x) = x^p$

$$\kappa_f(x) = \left| \frac{x(x^p)'}{x^p} \right| = \left| \frac{xp x^{p-1}}{x^p} \right| = \left| \frac{xp}{x} \right| = |p|$$

(b) $f(x) = \ln x$

$$\kappa_f(x) = \left| \frac{x(\ln x)'}{\ln x} \right| = \left| \frac{x * 1/x}{\ln x} \right| = \frac{1}{|\ln x|}$$

(c) $f(x) = \cos x$

$$\kappa_f(x) = \left| \frac{x(\cos x)'}{\cos x} \right| = \left| \frac{-x \sin x}{\cos x} \right| = |x \tan x|$$

(d) $f(x) = e^x$

$$\kappa_f(x) = \left| \frac{x(e^x)'}{e^x} \right| = |x|$$

Exercise (1.2.3)

Calculate the relative condition number of each function, and identify all values of x at which $\kappa_f(x) \rightarrow \infty$ (including limits as $x \rightarrow \pm\infty$).

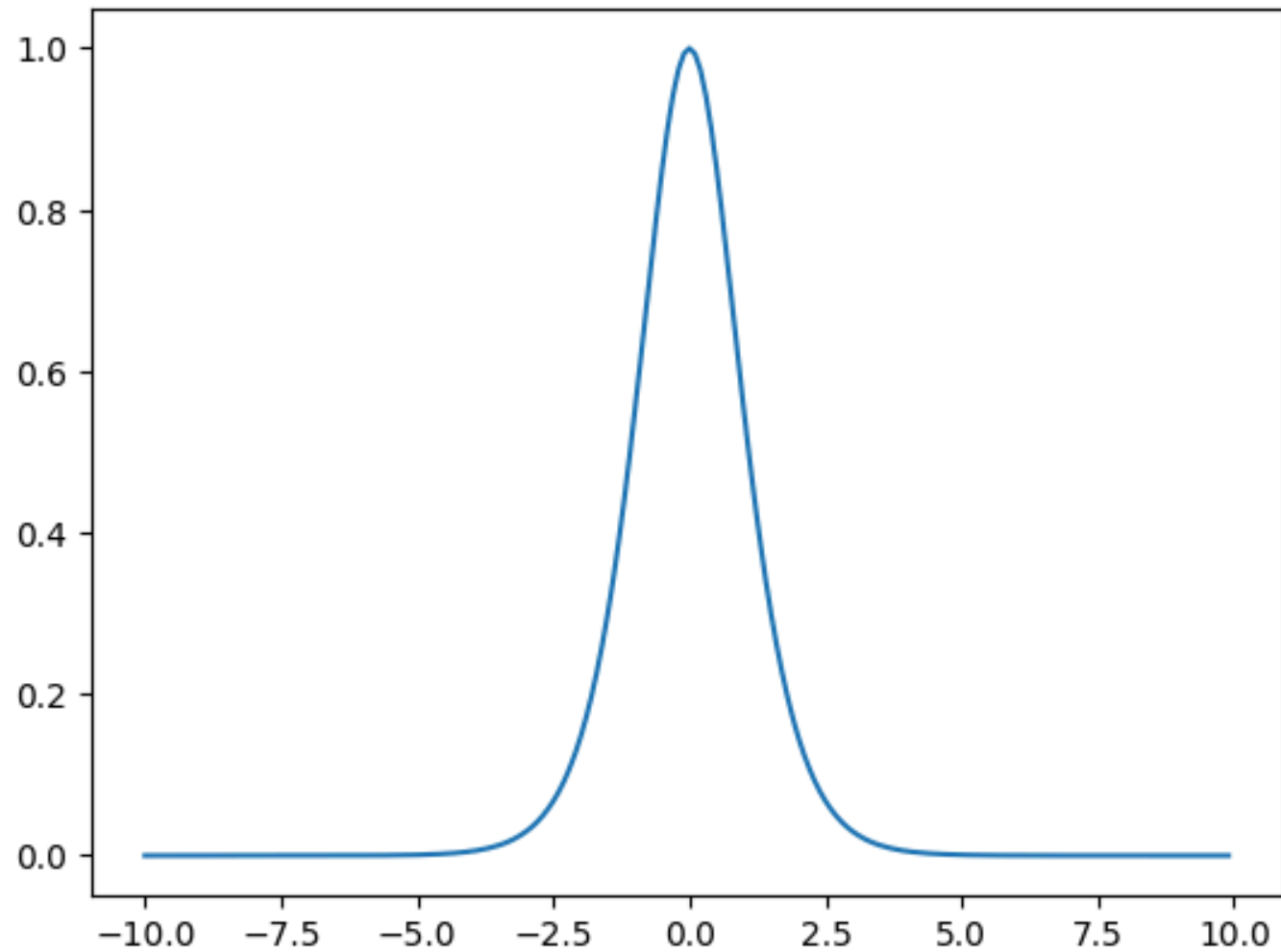
(a) $f(x) = \tanh x$

$$\kappa_f(x) = \left| \frac{x(\tanh x)'}{\tanh x} \right| = \left| \frac{x \operatorname{sech}^2 x}{\tanh x} \right| = \left| \frac{x}{\sinh x \cosh x} \right|$$

```
In [10]: import numpy as np
import matplotlib.pyplot as plt

x_axis = np.arange(-10, 10, 0.1)
plt.plot(x_axis, [abs(x / (np.sinh(x) * np.cosh(x))) for x in x_axis])
```

```
Out[10]: [<matplotlib.lines.Line2D at 0x2bca2137750>]
```



There are no values of x that tend to infinity.

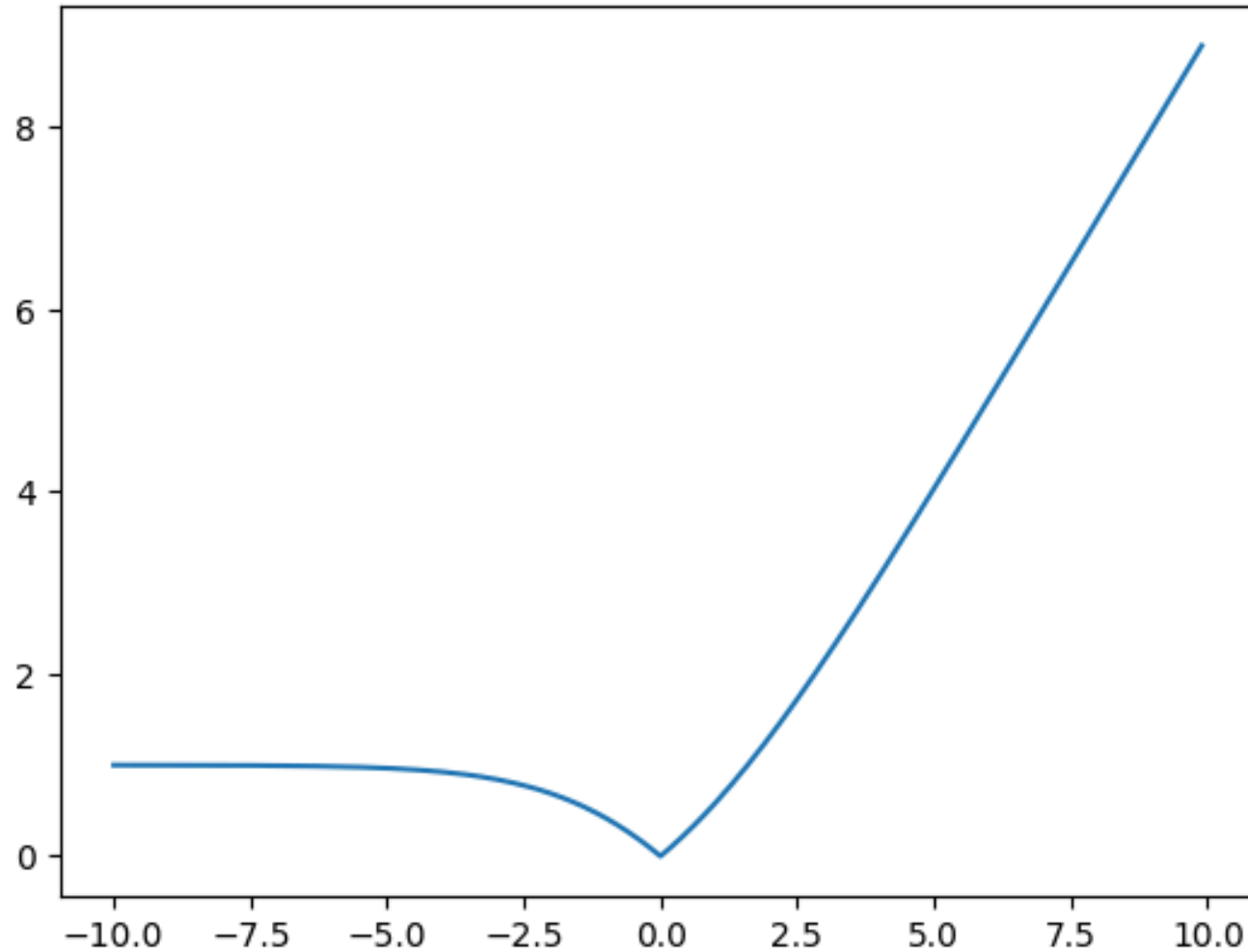
$$(b) f(x) = \frac{e^x - 1}{x}$$

$$\kappa_f(x) = \left| \frac{x(x^{-1}(e^x - 1))'}{x^{-1}(e^x - 1)} \right| = \left| \frac{e^x(x - 1) + 1}{e^x - 1} \right|$$

```
In [9]: import numpy as np
import matplotlib.pyplot as plt
```

```
x_axis = np.arange(-10, 10, 0.1)
plt.plot(x_axis, [abs((np.exp(x) * (x - 1) + 1) / (np.exp(x) - 1)) for x in x_axis])
```

Out[9]: [<matplotlib.lines.Line2D at 0x2bca20e8b90>]



As $x \rightarrow \infty$, $\kappa_f(x) \rightarrow \infty$.

(c) $f(x) = \frac{1 - \cos x}{x}$

$$\kappa_f(x) = \left| \frac{x(x^{-1}(1 - \cos x))'}{x^{-1}(1 - \cos x)} \right| = \left| \frac{x - 1}{\tan \frac{x}{2}} \right|$$

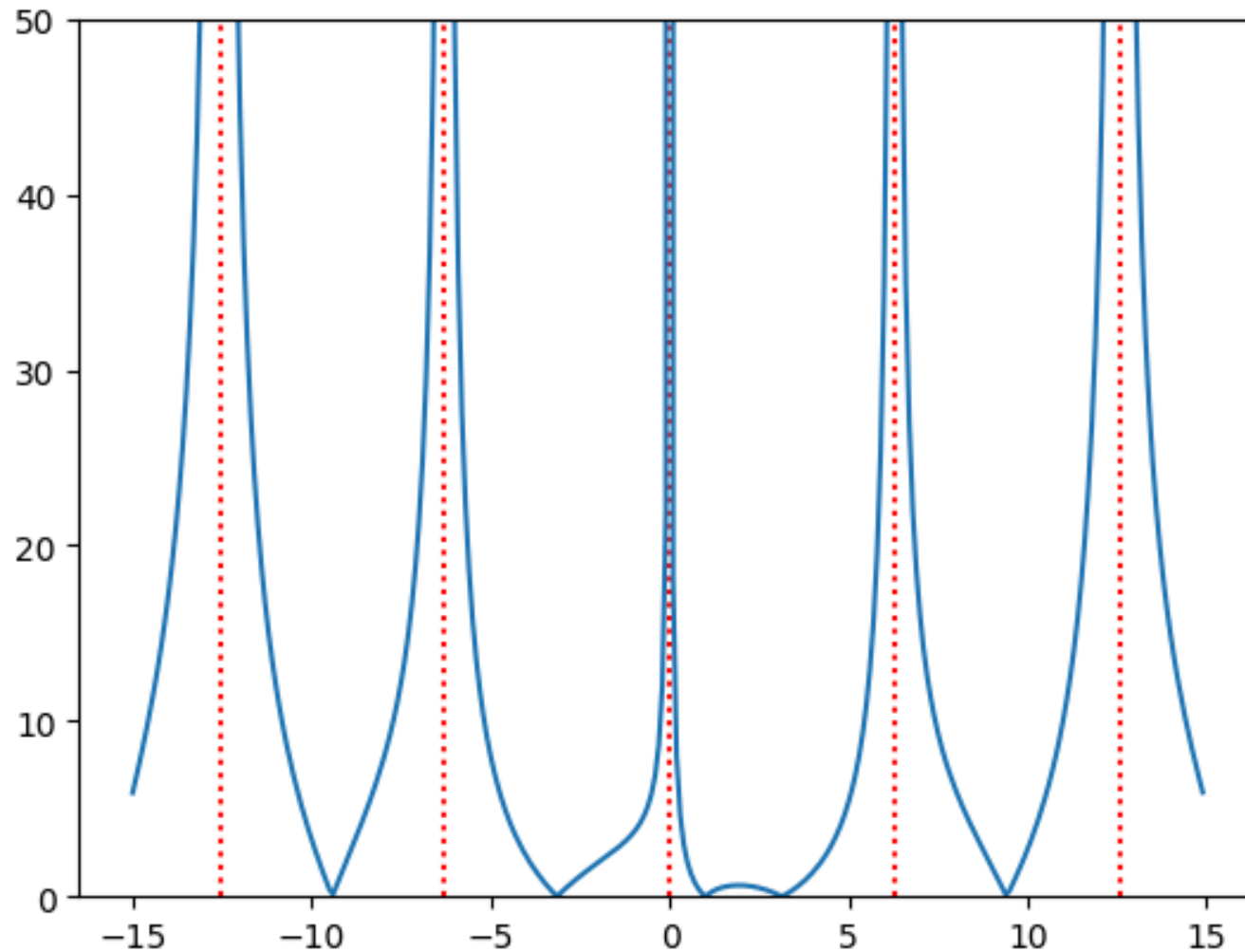
```

In [18]: import numpy as np
import matplotlib.pyplot as plt

x_axis = np.arange(-15, 15, 0.1)
plt.ylim(0, 50)
plt.vlines([c * 2 * np.pi for c in range(-2, 2 + 1)], 0, 50, "r", ":")
plt.plot(x_axis, [abs((x - 1) / np.tan(x / 2)) for x in x_axis])

```

Out[18]: [



$\kappa_f(x) \rightarrow \infty$ when $x = 2k\pi$ for integer k .

Exercise (1.2.8)

Generalize (1.2.13) to finding a root of the n th degree polynomial $p(x) = a_n x^n + \cdots + a_1 x + a_0$, and show that the relative condition number of a root r with respect to perturbations only in a_k is $\kappa_r(a_k) = \left| \frac{a_k r^{k-1}}{p'(r)} \right|$.

(1.2.13) states:

$$\kappa_f(a) = \left| \frac{a}{r_1} \cdot \frac{dr_1}{da} \right| = \left| \frac{ar_1}{2ar_1 + b} \right| = \left| \frac{r_1}{r_1 - r_2} \right|$$

For finding the conditioning of the leading coefficient of some quadratic $at^2 + bt + c = 0$ on some root $f(a) = r_1$.

For a general n th degree polynomial of the form $p(x) = a_n x^n + \cdots + a_1 x + a_0$, we can follow the same steps outlined in the textbook.

Differentiating $a_n r_1^n + \cdots + a_k r_1^k + \cdots + a_1 r_1 + a_0$ implicitly with respect to a_k , holding all other a_i fixed, yields

$$\begin{aligned} & \frac{d}{da_k} (a_n r_1^n + \cdots + a_k r_1^k + \cdots + a_1 r_1 + a_0) \\ &= a_n(n) r_1^{n-1} \frac{dr_1}{da_k} + \cdots + (r_1^k + a_k(k) r_1^{k-1} \frac{dr_1}{da_k}) + \cdots + a_1(1) r_1^0 \frac{dr_1}{da_k} \\ &= \frac{dr_1}{da_k} (a_n(n) r_1^{n-1} + \cdots + a_k(k) r_1^{k-1} + \cdots + a_1) + r_1^k \stackrel{\text{SET}}{=} 0 \\ &\rightarrow \frac{dr_1}{da_k} = \frac{-r_1^k}{a_n(n) r_1^{n-1} + \cdots + a_1} \end{aligned}$$

Now that we have a generalized equation for $\frac{dr_1}{da_k}$ and indeed any $\frac{dr}{da_k}$, we can find the condition number of a root r with respect to perturbations only in a_k .

$$\kappa_r(a_k) = \left| \frac{a_k(dr/da_k)}{r} \right| = \left| \frac{a_k}{r} \cdot \frac{-r_1^k}{a_n(n)r_1^{n-1} + \dots + a_1} \right| = \left| \frac{a_k r^{k-1}}{p'(r)} \right|$$

Exercise (1.4.2)

Let $f(x) = \frac{e^x - 1}{x}$.

(a) Find the condition number $\kappa_f(x)$. What is the maximum of $\kappa_f(x)$ over $-1 \leq x \leq 1$?

As calculated in 1.2.3b,

$$\kappa_f(x) = \left| \frac{e^x(x-1) + 1}{e^x - 1} \right|$$

By way of Desmos, the maximum of $\kappa_f(x)$ over $-1 \leq x \leq 1$ is at the right bound $\kappa_f(1) = \frac{1}{e-1}$.

(b) Use the "obvious" algorithm `(exp(x) - 1) / x` to compute $f(x)$ at $x = 10^{-2}, 10^{-3}, 10^{-4}, \dots, 10^{-11}$.

```
In [ ]: import numpy as np

x_values = [10 ** e for e in range(-2, -11 - 1, -1)]

def obvious(x):
    return (np.exp(x) - 1) / x

obvious_algorithm = []
for x in x_values:
    y = obvious(x)
    obvious_algorithm.append(y)
    print(f"f({x:e}) = {y}")
```

```

f(1.000000e-02) = 1.005016708416795
f(1.000000e-03) = 1.0005001667083846
f(1.000000e-04) = 1.000050001667141
f(1.000000e-05) = 1.000005000006965
f(1.000000e-06) = 1.0000004999621837
f(1.000000e-07) = 1.0000000494336803
f(1.000000e-08) = 0.999999993922529
f(1.000000e-09) = 1.000000082740371
f(1.000000e-10) = 1.000000082740371
f(1.000000e-11) = 1.000000082740371

```

(c) Create a second algorithm from the first 8 terms of the Maclaurin series, i.e.,

$$f(x) \approx p(x) = 1 + \frac{1}{2!}x + \frac{1}{3!}x^2 + \cdots + \frac{1}{8!}x^7.$$

Evaluate it at the same values of x as in part (b).

```

In [ ]: from math import factorial

def maclaurin(x):
    return sum([(1 / factorial(n)) * (x ** (n - 1)) for n in range(1, 8 + 1)])

maclaurin_algorithm = []
for x in x_values:
    y = maclaurin(x)
    maclaurin_algorithm.append(y)
    print(f"p({x:e}) = {y}")

```

```

p(1.000000e-02) = 1.0050167084168058
p(1.000000e-03) = 1.0005001667083417
p(1.000000e-04) = 1.0000500016667084
p(1.000000e-05) = 1.0000050000166667
p(1.000000e-06) = 1.0000005000001666
p(1.000000e-07) = 1.0000000500000017
p(1.000000e-08) = 1.0000000005
p(1.000000e-09) = 1.0000000005
p(1.000000e-10) = 1.00000000005
p(1.000000e-11) = 1.000000000005

```

(d) Make a table of the relative difference between the two algorithms as a function of x . Which algorithm is more accurate, and why?

```

In [12]: def relative_difference(x_tilda, x):
          return abs((x_tilda - x) / x)

          for i in range(len(x_values)):
              print(f"Relative difference for x = {x_values[i]:e}: {relative_difference(obvious_algorithm[i], r

```

```

Relative difference for x = 1.000000e-02: 1.0825875381182465e-14
Relative difference for x = 1.000000e-03: 4.283318501737311e-14
Relative difference for x = 1.000000e-04: 4.3252126360989365e-13
Relative difference for x = 1.000000e-05: 9.701746414095626e-12
Relative difference for x = 1.000000e-06: 3.798293112700397e-11
Relative difference for x = 1.000000e-07: 5.663214058761472e-10
Relative difference for x = 1.000000e-08: 1.1077470885147441e-08
Relative difference for x = 1.000000e-09: 8.22403709166e-08
Relative difference for x = 1.000000e-10: 8.269037099081884e-08
Relative difference for x = 1.000000e-11: 8.273537099826299e-08

```

The Maclaurin algorithm is more accurate. The obvious algorithm performs a subtraction between two nearly identical numbers, e^ϵ and 1, which has very poor conditioning. The Maclaurin algorithm avoids this. Using an evaluation at $x = 10^{-11}$ out to 64 decimal digits as the ground truth, the obvious algorithm gets 7.082 significant digits, whereas the Maclaurin algorithm gets *at least* 22; WolframAlpha actually thinks the relative error is 0 and can't take the log of it.

Exercise (1.4.3)

The function

$$x = \cosh(y) = \frac{e^y + e^{-y}}{2}$$

can be inverted to yield a formula for $\operatorname{acosh}(x)$:

$$y = \operatorname{acosh}(x) = -\ln(x - \sqrt{x^2 - 1}).$$

For the steps below, define $y_i = 4i$ and $x_i = \cosh(y_i)$ for $i = 1, \dots, 4$. Hence $y_i = \operatorname{acosh}(x_i)$.

(a) Find the relative condition number of evaluating $f(x) = \operatorname{acosh}(x)$. (You can use (1.4.4) or look up a formula for f' in a calculus book.) Evaluate κ_f at all the x_i . (You will find that κ is less than 20 at these inputs, so the conditioning is fine.)

$$\kappa_f(x) = \left| \frac{xf'(x)}{f(x)} \right| = \left| \frac{x}{\ln(x - \sqrt{x^2 - 1})\sqrt{x^2 - 1}} \right|$$

$$\kappa_f(x_1) = \kappa_f\left(\frac{e^4 + e^{-4}}{2}\right) = 4$$

$$\kappa_f(x_2) = \kappa_f\left(\frac{e^8 + e^{-8}}{2}\right) = 8$$

$$\kappa_f(x_3) = \kappa_f\left(\frac{e^{12} + e^{-12}}{2}\right) = 12$$

$$\kappa_f(x_4) = \kappa_f\left(\frac{e^{16} + e^{-16}}{2}\right) = 16$$

(b) Use (1.4.4) to approximate $f(x_i)$ for all i . Compute the relative accuracy of the results. Why are some of the results so inaccurate?

```
In [19]: import numpy as np

def cosh(y):
    return (np.exp(y) + np.exp(-1 * y)) / 2

def f(x):
    return -1 * np.log(x - np.sqrt(x ** 2 - 1))

for i in range(1, 4 + 1):
    y = 4 * i
    x = cosh(y)
    f_x = f(x)
    print(f"f({x}) = {f_x}; RE = {abs((f_x - y) / y)}")
```

```
f(27.308232836016487) = 4.000000000000046; RE = 1.1546319456101628e-14
f(1490.479161252178) = 8.00000000017109; RE = 2.138622612335439e-11
f(81377.39571257407) = 12.000000137072186; RE = 1.1422682207514375e-08
f(4443055.260253992) = 15.998624871201619; RE = 8.594554989882486e-05
```

The results are so inaccurate because $(x_i)^2$, used by the naive `acosh` formula, can get supremely massive; $(x_4)^2$ is something like $2 \cdot 10^{13}$. Floating points cannot accurately represent supremely massive irrational numbers, so scaling them down with `acosh` can show the precision errors seen above.

(c) An alternative formula is

$$y = 2 \log \left(\sqrt{\frac{x+1}{2}} + \sqrt{\frac{x-1}{2}} \right).$$

Apply (1.4.5) to approximate $f(x_i)$ for all i , again computing the relative accuracy of the results.

```
In [20]: def f_alternative(x):
    return 2 * np.log(np.sqrt((x + 1) / 2) + np.sqrt((x - 1) / 2))

for i in range(1, 4 + 1):
    y = 4 * i
    x = cosh(y)
```

```
f_x = f_alternative(x)
print(f"f({x}) = {f_x}; RE = {abs((f_x - y) / y)}")
```

f(27.308232836016487) = 4.0; RE = 0.0

f(1490.479161252178) = 8.0; RE = 0.0

f(81377.39571257407) = 12.0; RE = 0.0

f(4443055.260253992) = 16.0; RE = 0.0

Hooray!